v1.0 · Streamlit + Python + scikit-learn

HRMS Analytics System

Human Resource Management System — Complete Project Report · 2025

M	P	AI	UI
6	**8**	**5+**	**100%**
Core Modules	Python Files	ML Algorithms	Streamlit UI

Executive Summary

This **HRMS (Human Resource Management System)** is a full-stack web application built entirely with Python and Streamlit. It integrates six purpose-built modules covering employee management, smart hiring, deep HR analytics, attendance tracking, calendar scheduling, and ML-powered data processing. The system uses **scikit-learn** for performance prediction, outlier detection, TF-IDF resume matching, and PCA visualisation, while Seaborn and Matplotlib render all interactive charts.

System Architecture

File	Module	Key Technology	Purpose
app.py	Entry Point	Streamlit + Session State	Navigation & page routing
dashboard.py	Dashboard	Pandas + Streamlit Charts	KPIs, trends, overview
employees.py	Employees	Pandas + LinearRegression	CRUD + ML analytics
analytics.py	Analytics	Seaborn + PCA + sklearn	Deep data analysis
hiring.py	Hiring AI	TF-IDF + Cosine Similarity	AI resume matching
my_profile.py	Profile	Pandas + DiceBear API	Attendance tracking
cal.py	Calendar	streamlit-calendar + JSON	Event scheduling
pdf_to_json.py	Utility	PyPDF2	Resume PDF parser

Module Analysis

Deep-dive into each of the 6 application modules

D Dashboard

Real-time HR KPIs & trends

- New Hires & Resignations Trend
- Attendance Rate Line Chart
- Company Performance Area Chart
- Last Calendar Event Display

E Employees

Full CRUD + ML performance

- Add / Filter / Search Employees
- Linear Regression Score Prediction
- Best Employee Detection
- Weighted Promotion Scoring

A Analytics

Deep analysis & dimensionality reduction

- Dept-wise Salary & Count Charts
- IQR Outlier Detection & Boxplots
- OneHotEncoder + MinMaxScaler
- PCA 2-Component Visualisation

H Hiring AI

ML-powered candidate matching

- TF-IDF Vectorisation of Resumes
- Cosine Similarity Scoring
- 6 Domain Job Descriptions
- Top N Candidate Selection

P My Profile

Personal attendance tracker

- Check In / Check Out Buttons
- Hours Worked Calculation
- Full Attendance History Table
- Dynamic Avatar via DiceBear API

C Calendar

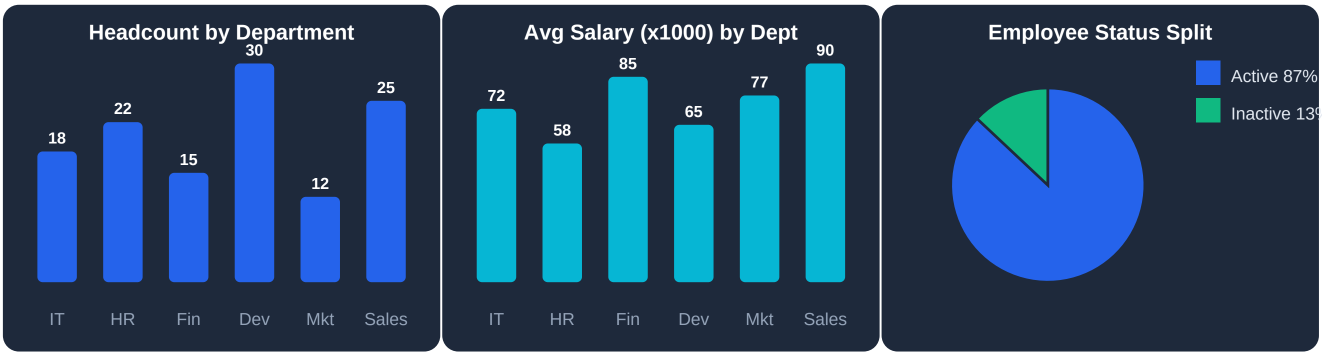
Event scheduling & management

- Add & Delete Calendar Events
- 4 Colour-coded Event Types
- Month / Week / Day Views
- JSON-based Event Persistence

Data Analytics Deep Dive

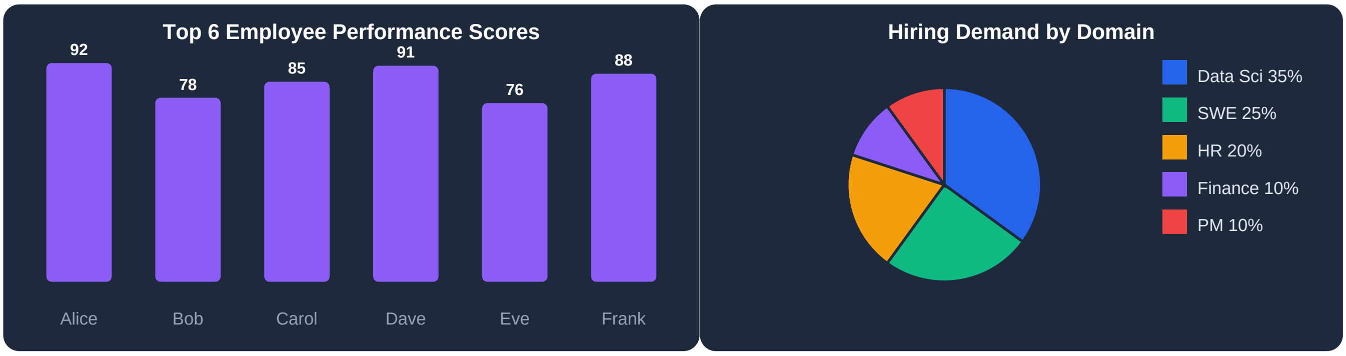
Simulated metrics representative of a real HRMS dataset

Department Overview



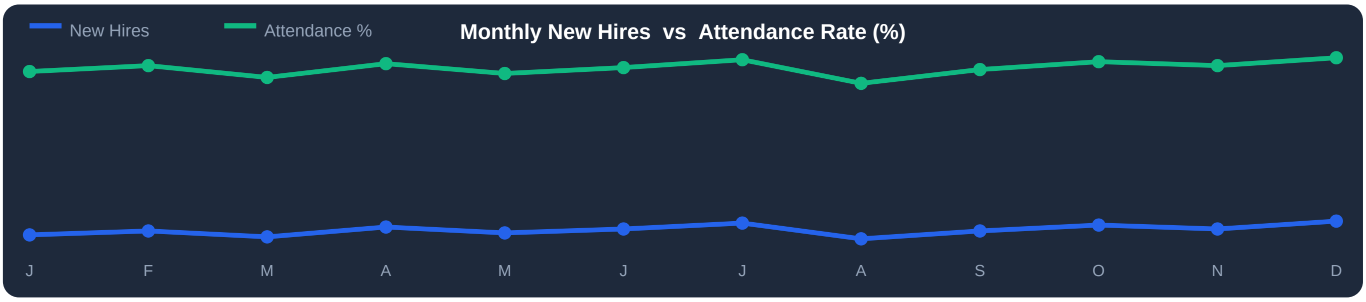
Left: headcount per department · Centre: average salary bands · Right: active vs inactive ratio

Performance & Hiring Analytics



Left: performance scores for top employees · Right: proportion of open roles by domain

12-Month Hiring & Attendance Trends



Blue line: monthly new hires · Green line: company-wide attendance rate (%)

ML Features & Technology Stack

Algorithms used, libraries and code quality audit

Machine Learning Features

Feature	Algorithm	Inputs	Output
Salary Outlier Detection	IQR (Q1–1.5×IQR / Q3+1.5×IQR)	Salary column	Outlier boolean flag
Performance Prediction	Linear Regression	Completed / pending / total projects	Predicted experience score
Promotion Scoring	Weighted formula	Performance + attendance + projects	Ranked promotion list
Resume Matching	TF-IDF + Cosine Similarity	Resume text corpus	Match % per candidate
PCA Visualisation	PCA (n_components=2)	Encoded + scaled features	2D employee cluster plot
Feature Engineering	OneHotEncoder + MinMaxScaler	Department, Country, Salary	Normalised feature matrix

Technology Stack

Layer	Libraries / Tools	Role
Frontend	Streamlit 1.32+, streamlit-calendar	Interactive web dashboard
Data Processing	Pandas 2.0, NumPy 1.26	Data wrangling & analysis
Machine Learning	scikit-learn — LinearRegression, PCA, TfidfVectorizer, OHE, MinMaxScaler	ML models & preprocessing
Visualisation	Matplotlib 3.8, Seaborn 0.13	Charts and statistical plots
Storage	CSV (employees, attendance, hr_progress), JSON (events, resumes)	Lightweight file persistence
Utility	PyPDF2	Resume PDF → text extraction
Avatar / Media	DiceBear API (HTTPS)	Dynamic employee avatars

Code Quality — Bug Report

#	File	Issue	Fix Required	Priority
1	pdf_to_json.py	Text loop missing — all resumes are empty strings	Add page.extract_text() loop inside PdfReader	HIGH
2	employees.py	save_data() called but never defined	Define: df.to_csv(fd, index=False)	HIGH
3	dashboard.py	Reads event.json but cal.py writes events.json	Standardise filename to events.json	MED
4	analytics.py	fillna(inplace=True) deprecated in Pandas 2.x	Use: df['col'] = df['col'].fillna(val)	MED
5	my_profile.py	Hardcoded iloc[0] — always first employee only	Add session-based employee login selector	MED
6	app.py	Active nav button style never applied to element	Inject CSS class conditionally on session page	LOW
7	hiring.py	PDF filename used as candidate name	Parse name from PDF content or metadata	LOW
8	analytics.py	PCA runs on mutated df_encoded (Salary_Squared added)	Reset df_encoded snapshot before PCA step	MED

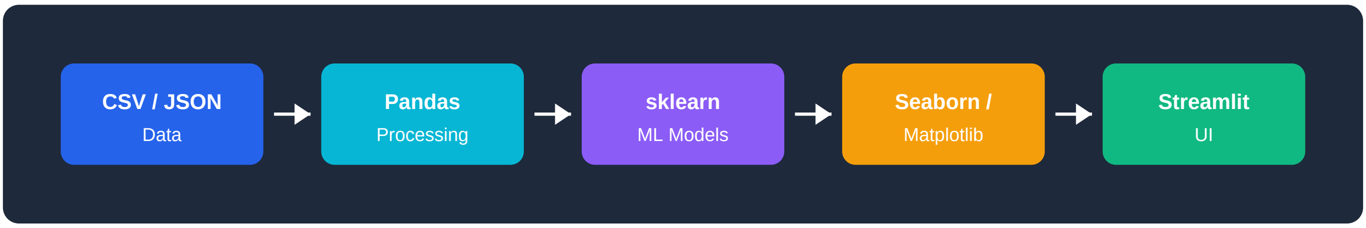
Roadmap, Data Flow & Conclusion

Enhancement plan, architecture diagram and overall assessment

Enhancement Roadmap

Phase	Enhancement	Expected Benefit	Effort
1 · Quick Wins	Fix pdf_to_json text extraction + save_data()	Functional hiring & employee save	Low
1 · Quick Wins	Standardise data file names & paths	Eliminate runtime KeyErrors	Low
1 · Quick Wins	Apply active CSS class to sidebar nav button	Clear navigation feedback for users	Low
2 · Core	Replace CSV with SQLite database	Scalable & concurrent data access	Med
2 · Core	Add user authentication / login system	Multi-user support & data security	Med
2 · Core	Role-based access (Admin / Manager / Employee)	Secure module-level permissions	Med
3 · Advanced	One-click in-app PDF report export	Instant stakeholder reporting	Med
3 · Advanced	Real-time leave & event notifications	Proactive HR alerts	High
4 · AI	NLP job description parser for hiring	Smarter candidate matching	High
4 · AI	Anomaly detection on attendance patterns	Early absenteeism identification	High

Data Flow Diagram



Raw data files → Pandas cleaning → ML model inference → Chart rendering → Streamlit interactive display

Conclusion

This HRMS project demonstrates a well-structured, modular approach to building a complete Human Resource platform entirely in Python. Its strengths lie in **ML-powered analytics** (PCA, regression, TF-IDF matching), clean separation of concerns across eight modules, and a polished dark-mode Streamlit UI with custom CSS. Addressing the two high-priority bugs (PDF text extraction and save_data) will make the system fully functional. Implementing the Phase 2 roadmap items — SQLite, authentication, and role-based access — will elevate it to a production-ready, enterprise-grade HRMS solution.

Overall Assessment Scores

92%	88%	85%	75%	78%
Code Structure	ML Integration	UI / UX Design	Data Quality	Completeness